

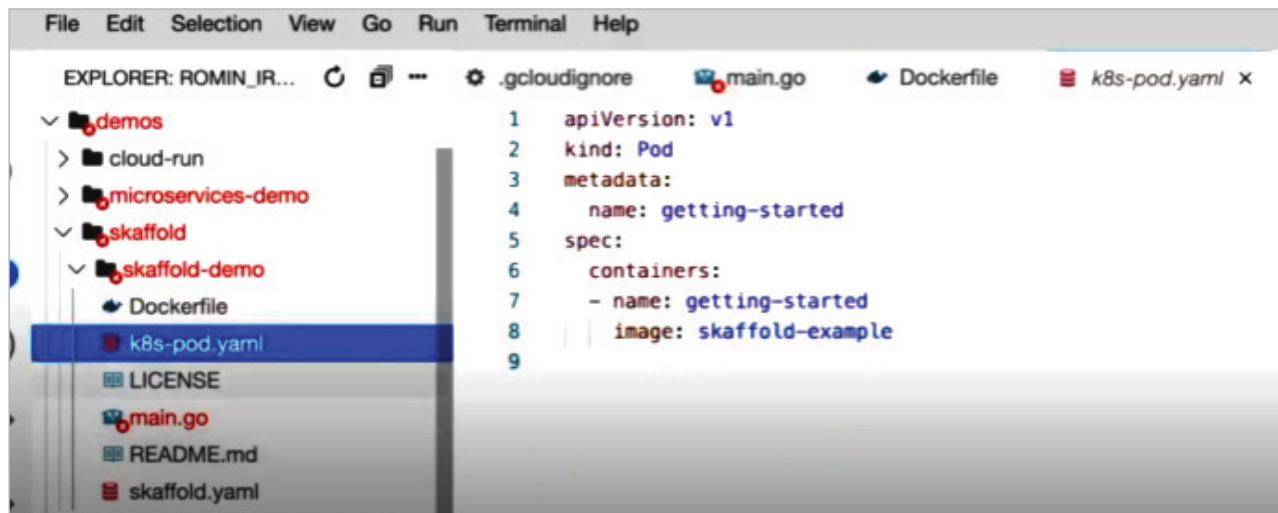


Figure 5: *k8s-pod.yaml*

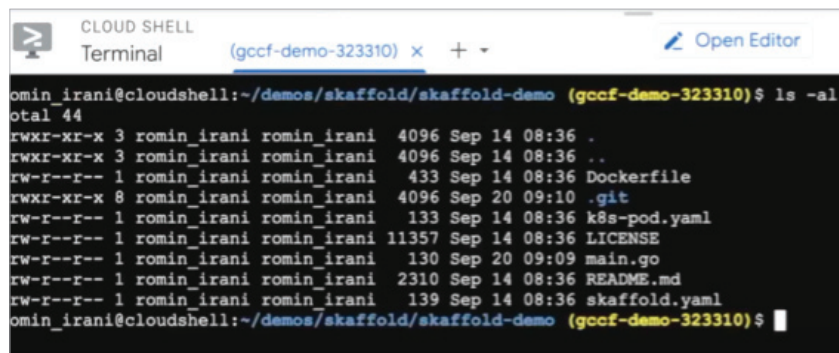


Figure 6: Inside the Cloud Shell terminal

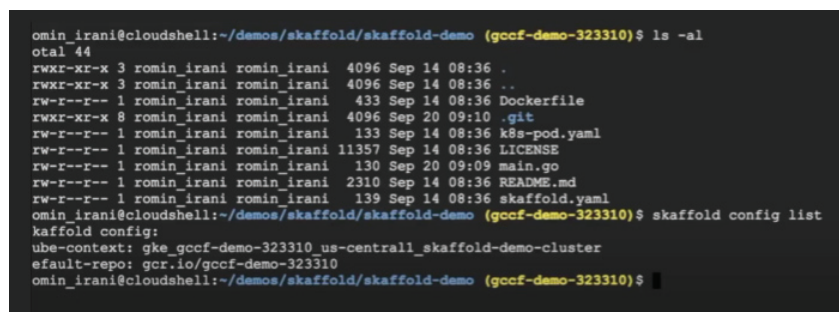


Figure 7: '\$skaffold config list' command is applied in the terminal

a build perspective, as Skaffold really knows what it is supposed to build.

In the *pod.yaml* file (Figure 5), it is just stated that the image that is needed to be created is *Skaffold-example*. The *Skaffold.yaml* file will also show you the image that is needed to be built along with the deployment steps. All these steps will let Skaffold

pretty much understand and detect the configuration.

In Figure 6, you can see that in the terminal inside the demo folder we have the *Skaffold.yaml* files, the *k8s-pod.yaml* and the *main.go*. Since the Skaffold tool is already installed as part of Google Cloud Shell, on typing *\$skaffold config list* you can see (Figure 7) that the

current kube context is set to that demo cluster, and the default-repo is also set to the Google container registry.

Now, we have a folder in which we have got our configuration file. All we need to do is run the developer end-to-end pipeline in Skaffold by using the *\$skaffold dev* command. It will build everything up for the first time, and will begin with the deployment process. It will then provide you the relevant output and will wait for the code changes as well.

Skaffold is just like any other command-line tool that can be deployed on your machine or any other server that you desire; it comes preconfigured with Google Cloud Shell. All in all, Skaffold can automate a lot of steps that you would need to go through manually otherwise. Automating these steps will not only save time so that you can focus on other important things, but also make the application less prone to unwanted bugs and errors. **END** 🐧

By: Romin Irani

The author is developer advocate, Google Cloud.

Transcribed and curated by Laveesh Kocher.